# Manual Complet de Teoria Grafurilor

Sinteză Extinsă

## Cuprins


**1 1. Structura și Conexitatea** **2**
- 1.1 Algoritmul Havel-Hakimi . . . 2
- 1.2 Parcurgeri: DFS vs BFS . . . 2
- 1.3 Clasificarea Muchiilor în DFS . . . 2
- 1.4 Componente Tare Conexe (Kosaraju) . . . 2
- 1.5 Muchii Critice (Punți) . . . 3

**2 2. Arbori Parțiali de Cost Minim (APCM)** **3**
- 2.1 Algoritmul Kruskal . . . 3
- 2.2 Algoritmul Prim . . . 3

**3 3. Drumuri Minime** **3**
- 3.1 Algoritmul Dijkstra . . . 3
- 3.2 Algoritmul Bellman-Ford . . . 4
- 3.3 Algoritmul Floyd-Warshall . . . 4

**4 4. Fluxuri și Cuplaje** **4**
- 4.1 Flux Maxim (Max Flow) . . . 4
- 4.2 Algoritmul Edmonds-Karp . . . 4
- 4.3 Cuplaj Maxim în Graf Bipartit . . . 4

**5 5. Grafuri Speciale** **5**
- 5.1 Eulerian vs Hamiltonian . . . 5
- 5.2 Grafuri Planare . . . 5

**6 6. Programare Dinamică: Distanța de Editare** **5**

# 1 1. Structura și Conexitatea

## 1.1 Algoritmul Havel-Hakimi

**Intuiție / Logica din spate**

Gândește-te la gradele nodurilor ca la niște "brațe" libere. Algoritmul spune: "Ia nodul cu cele mai multe brațe (șeful) și conectează-l cu următorii cei mai dotați". Dacă rămâi cu brațe negative (imposibil), nu se poate forma graful.

**Algoritm: Havel-Hakimi**

Problema: Se dă o secvență de grade $S$. Putem construi un graf simplu?

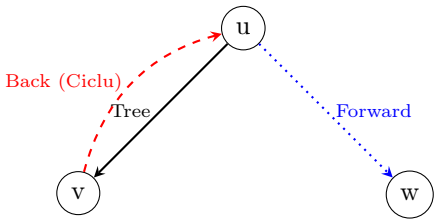
1. Sortează secvența descrescător: $d_1 \geq d_2 \geq \cdots \geq d_n$.
2. Dacă $d_1 > n - 1$ sau apare un număr negativ $\rightarrow$ IMPOSIBIL.
3. Dacă toate sunt 0 $\rightarrow$ POSIBIL (STOP).
4. Elimină $d_1$. Scade câte 1 din următoarele $d_1$ numere din listă.
5. Repetă de la pasul 1.

## 1.2 Parcurgeri: DFS vs BFS

- **BFS (Lățime):** Ca o undă într-un lac. Găsește cel mai scurt drum (ca număr de muchii). Folosește o **Coadă**.
- **DFS (Adâncime):** Ca un explorator într-un labirint care merge până se lovește de un zid, apoi se întoarce puțin (backtracking). Folosește o **Stivă**.

## 1.3 Clasificarea Muchiilor în DFS

Când parcurgem graful în adâncime, muchiile care NU fac parte din arborele DFS ne spun lucruri despre structura grafului.




**Intuiție / Logica din spate**

- **Tree Edge:** Drumul normal pe care îl descoperim.
- **Back Edge (Roșu):** O "scurtătură" înapoi spre un strămoș. **Aceasta demonstrează existența unui CICLU.**

## 1.4 Componente Tare Conexe (Kosaraju)

**Intuiție / Logica din spate**

Ideea genială: Dacă poți ajunge de la A la B, și în graful cu sensurile inversate poți ajunge tot de la A la B (care de fapt e B la A în original), înseamnă că există un ciclu A-B-A.

**Algoritm: Kosaraju (Complexitate V+E)**

1. Fă DFS pe graful $G$ și reține nodurile într-o **stivă** în ordinea finalizării.
2. Construiește graful transpus $G^T$ (inversează toate săgețile).
3. Extrage nodurile din stivă. Dacă nodul curent nu e vizitat, pornește un nou DFS în $G^T$.
4. Toate nodurile atinse într-un astfel de DFS formează o Componentă Tare Conexă (CTC).

### 1.5 Muchii Critice (Punți)

O muchie $(u, v)$ e critică dacă e singura cale de a ajunge la $v$ (eliminarea ei rupe conexitatea).

**Algoritm: Detecție Muchii Critice**

Calculăm pentru fiecare nod $u$:

- $niv[u]$: Nivelul în arborele DFS.
- $low[u]$: Cel mai mic nivel la care se poate ajunge folosind o muchie de întoarcere din subarborele lui $u$.

**Condiție:** Muchia $(u, v)$ este critică dacă $low[v] > niv[u]$ (adică din $v$ nu te poți întoarce "mai sus" de $u$).

## 2 2. Arbori Parțiali de Cost Minim (APCM)

**Intuiție / Logica din spate**

Scopul: Să conectăm toate orașele cu cost total minim de asfaltare (fără a crea cicluri inutile).

### 2.1 Algoritmul Kruskal

- **Strategie:** "Lacom după muchii".
- Sortează toate muchiile crescător după cost.
- Iterează și adaugă muchia doar dacă nu unește două noduri care sunt deja în aceeași componentă (folosind *Disjoint Sets* / Union-Find).

### 2.2 Algoritmul Prim

- **Strategie:** "Lacom după vecini".
- Pornește dintr-un nod arbitrar (zona asfaltată).
- La fiecare pas, alege cea mai ieftină muchie care iese din zona asfaltată către un nod nevizitat.
- Folosește *Priority Queue*.

## 3 3. Drumuri Minime

### 3.1 Algoritmul Dijkstra

**Intuiție / Logica din spate**

Funcționează ca apa care curge prin țevi: ajunge mai întâi la nodurile apropiate. Relaxează vecinii imediat. **Limitare:** Nu funcționează corect dacă există muchii cu cost negativ!

**Relaxarea:**

$$\text{Dacă } d[u] + w(u, v) < d[v] \Rightarrow d[v] = d[u] + w(u, v)$$

### 3.2 Algoritmul Bellman-Ford

**Intuiție / Logica din spate**

Este mai lent, dar sigur. Relaxează *toate* muchiile de $N-1$ ori. Funcționează și cu costuri negative.

**Detecție Ciclu Negativ:** Dacă la pasul $N$ se mai poate face o relaxare, înseamnă că există un ciclu infinit de scădere a costului.

### 3.3 Algoritmul Floyd-Warshall

Programare dinamică. Calculează drumul minim între **oricare două noduri**.

$$d[i][j] = \min(d[i][j], \quad d[i][k] + d[k][j])$$

Unde $k$ este un nod intermediar permis.

## 4 4. Fluxuri și Cuplaje

### 4.1 Flux Maxim (Max Flow)

**Intuiție / Logica din spate**

Gândește-te la o rețea de țevi. Câtă apă intră în Sursă ($S$) trebuie să iasă în Destinație ($T$), respectând capacitatea țevilor. **Teorema Min-Cut:** Dacă tai rețeaua în două (separând $S$ de $T$), capacitatea totală a țevilor tăiate (dinspre $S$ spre $T$) este egală cu fluxul maxim. Gâtuirea limitează sistemul.

### 4.2 Algoritmul Edmonds-Karp

**Algoritm: Edmonds-Karp**

Este varianta Ford-Fulkerson în care alegem mereu drumul de ameliorare cu **cele mai puține muchii** (folosind BFS) în graful rezidual.

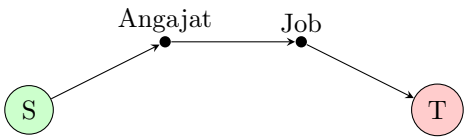
1. Caută drum $S \to T$ în graful rezidual cu BFS.
2. Calculează capacitatea minimă pe acel drum (cât flux putem împinge).
3. Actualizează fluxul (adaugă pe muchia directă, scade pe cea inversă).
4. Repetă până nu mai găsești drumuri.

### 4.3 Cuplaj Maxim în Graf Bipartit

**Intuiție / Logica din spate**

Problema "angajați pe posturi". Se rezolvă cu Flux Maxim:

- Creează o **Super-Sursă** legată de toți angajații.
- Creează o **Super-Destinație** legată de toate posturile.
- Toate capacitățile sunt 1.
- Fluxul Maxim = Numărul Maxim de Perechi (Cuplaje).



# 5 5. Grafuri Speciale

## 5.1 Eulerian vs Hamiltonian

Diferența Esențială

- **Eulerian (Muchii):** Vrei să parcurgi toate muchiile o singură dată (Poștașul).
- **Hamiltonian (Noduri):** Vrei să vizitezi toate orașele o singură dată (Comis-Voiajor).

- **Condiție Eulerian:** Graful să fie conex și toate nodurile să aibă **grad PAR**.
- **Condiție Hamiltonian:** Problemă grea (NP-Complet). *Teorema Dirac:* Dacă $n \geq 3$ și orice nod are grad $\geq n/2$, atunci e Hamiltonian.

## 5.2 Grafuri Planare

Grafuri care pot fi desenate fără intersecții de muchii. **Formula lui Euler:** $V - E + F = 2$ (unde $F$ include fața exterioară infinită).

# 6 6. Programare Dinamică: Distanța de Editare

**Intuiție / Logica din spate**

Cât ne costă să transformăm cuvântul "ANAF" în "BANANA"? Putem: Șterge, Insera sau Înlocui o literă.

**Algoritm: Recurența Levenshtein**

Fie $DP[i][j]$ costul pentru a transforma prefixul $A[1..i]$ în $B[1..j]$.

$$DP[i][j] = \min \begin{cases} DP[i-1][j] + 1 & \text{(Ștergere)} \\ DP[i][j-1] + 1 & \text{(Inserare)} \\ DP[i-1][j-1] + cost & \text{(Substituție)} \end{cases}$$

Unde $cost = 0$ dacă literele sunt identice, și 1 dacă sunt diferite.